

Laboratorio 6

Información

Integrantes:

Name	Institution ID	GitHub User
Josué Say	22801	JosueSay
Carlos Valladares	221164	vgcarlol

- [Repositorio](#)

Preparación de entorno

```
In [ ]: # %pip install -r requirements.txt
# jupyter nbconvert Lab6.ipynb --to html
```

Task 1

Como Ingeniero Principal (Lead AI Engineer) del proyecto AgriTech, usted debe justificar las decisiones arquitectónicas ante su equipo y sus clientes. Responda a los siguientes escenarios en su reporte (máximo 1 página por respuesta), combinando la teoría matemática con el pragmatismo laboral.

Inciso 1

Un desarrollador junior de su equipo sugiere:

"Para detectar con mayor precisión las texturas de las hojas enfermas, deberíamos construir una red secuencial clásica (tipo VGG) pero de 150 capas. Más profundo siempre es mejor".

Como líder técnico, explíquelo argumentativamente por qué esta red fracasará estrepitosamente en el entrenamiento (mencionando el fenómeno de degradación y el desvanecimiento del gradiente). Luego, justifique cómo la adición estructural de las conexiones residuales ($F(x) + x$) de ResNet rescata el proyecto, haciendo viable entrenar redes ultra-profundas sin colapsar.

La propuesta de una VGG secuencial de 150 capas parte de una idea incompleta porque más profundidad no garantiza mejor aprendizaje. En redes planas muy profundas aparece el

desvanecimiento del gradiente el cual al multiplicarse muchas derivadas en backpropagation, el gradiente tiende a cero y las primeras capas dejan de aprender.

Más grave aún es la degradación. ResNet mostró que al aumentar la profundidad en arquitecturas planas no solo puede empeorar la generalización, sino incluso el error de entrenamiento. Esto no es overfitting, sino un problema de optimización: aunque una red profunda podría imitar a una más pequeña dejando capas como identidad, el optimizador no logra encontrar esa solución. La red se vuelve más difícil de entrenar, no más efectiva.

ResNet soluciona esto reformulando el problema. En lugar de aprender $H(x)$ directamente, cada bloque aprende el residuo:

$$F(x) = H(x) - x$$

y la salida se define como:

$$y = F(x) + x$$

Si la transformación adicional no aporta valor, basta con que $F(x) \rightarrow 0$ y el bloque implementa identidad fácilmente.

En retropropagación aparece la clave:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial H} \left(\frac{\partial F}{\partial x} + 1 \right)$$

Ese $+1$ crea un camino directo para el gradiente, reduciendo el desvanecimiento y evitando la degradación.

Por lo que una VGG de 150 capas fracasaría por problemas de optimización. Las conexiones residuales hacen viable entrenar redes ultra-profundas porque estabilizan el flujo del gradiente y convierten la profundidad en una ventaja real, no en un obstáculo.

Inciso 2

Las enfermedades en las hojas de mango son visualmente heterogéneas: La Antracnosis se presenta como puntos negros diminutos, mientras que el Moho Polvoriento cubre áreas enormes de la hoja. Analice cómo la topología en paralelo del módulo Inception (usando filtros 3×3 y 5×5 simultáneamente) es ideal para este problema biológico en particular. Además, desde una perspectiva de costos de infraestructura (uso de GPUs en AWS o Google Cloud), explique cómo la inserción estratégica de convoluciones de 1×1 evita la explosión de la dimensionalidad y salva el presupuesto mensual de la startup.

Las enfermedades de la hoja de mango no aparecen en una sola escala. La **Antracnosis** genera puntos pequeños y localizados, mientras que el **Moho Polvoriento** cubre zonas amplias. Por eso, usar un único tamaño de filtro sería una decisión rígida para un problema que es claramente **multi-escala**.

El módulo **Inception** resuelve esto con su estructura en paralelo. En lugar de aplicar un solo filtro, procesa la misma imagen al mismo tiempo con filtros 3×3 y 5×5 (entre otros) y luego concatena los resultados. Así, la red puede detectar detalles pequeños y, a la vez, patrones más grandes dentro del mismo bloque.

- Los filtros 3×3 son adecuados para capturar puntos pequeños, bordes cortos y cambios locales de textura, como los de la Antracnosis.
- Los filtros 5×5 integran una región más amplia y detectan manchas extendidas o coberturas grandes, como el Moho Polvoriento.

La arquitectura no obliga a elegir entre detalle fino o contexto amplio: aprende ambos en paralelo.

Ahora bien, aplicar directamente filtros 3×3 y 5×5 sobre muchos canales sería muy costoso. El costo aproximado de una convolución es:

$$k^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$$

Al pasar de 3×3 a 5×5 , el factor espacial sube de 9 a 25. Si además hay muchos canales, el número de operaciones crece rápido y el uso de memoria también.

Aquí entran las convoluciones 1×1 , que actúan como reductores de canales antes de aplicar los filtros grandes. Primero comprimen la información y luego se ejecutan las ramas costosas. Si reducimos de C_{in} a C_r canales (con $C_r \ll C_{in}$), el costo baja mucho:

Sin reducción:

$$25 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$$

Con reducción previa:

$$1 \cdot C_{in} \cdot C_r \cdot H \cdot W + 25 \cdot C_r \cdot C_{out} \cdot H \cdot W$$

La diferencia es grande cuando C_r es mucho menor que C_{in} .

Desde el punto de vista de infraestructura, menos operaciones significan menos tiempo por época, menor uso de memoria y menor necesidad de GPUs grandes. En proveedores cloud eso se traduce directamente en menos costo mensual. Sin las 1×1 , el modelo podría volverse demasiado caro para una startup.

Inciso 3

El cliente final (el agricultor) usará la aplicación en un teléfono Android de gama baja en medio del campo, sin conexión a internet. Sabemos que MobileNet logra esta eficiencia gracias a la Depthwise Separable Convolution. Describa brevemente cómo esta convolución divide el trabajo (filtrado espacial vs. combinación de canales). Sin embargo, en ingeniería no hay soluciones mágicas, todo tiene trade-offs.

El costo aproximado de una convolución tradicional es:

$$k^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$$

donde el filtro de tamaño $k \times k$ opera sobre todos los canales de entrada para producir todos los canales de salida. Cuando C_{in} y C_{out} crecen, el número de operaciones aumenta rápidamente, lo que impacta tiempo de inferencia y consumo de memoria.

MobileNet reduce este costo usando **Depthwise Separable Convolution**, que divide el trabajo en dos etapas.

Primero aplica la **Depthwise Convolution**, que realiza solo el **filtrado espacial**. Se usa un filtro por cada canal de entrada, sin mezclar información entre canales. Su costo es:

$$k^2 \cdot C_{in} \cdot H \cdot W$$

Aquí se detectan bordes, texturas o patrones locales dentro de cada canal, pero no hay combinación entre ellos.

Luego aplica la **Pointwise Convolution** (1×1), que se encarga de la **combinación de canales**. Esta etapa aprende cómo integrar las características obtenidas en cada canal. Su costo es:

$$C_{in} \cdot C_{out} \cdot H \cdot W$$

El costo total queda:

$$k^2 \cdot C_{in} \cdot H \cdot W + C_{in} \cdot C_{out} \cdot H \cdot W$$

Comparado con la convolución estándar, la reducción es grande, especialmente cuando $k = 3$. En la práctica, el número de operaciones puede disminuir entre 8 y 9 veces. Eso se traduce en menor latencia, menor consumo de batería y modelos más pequeños, algo clave para ejecutar inferencia directamente en el teléfono del agricultor.

Task 2

Utilice PyTorch o TensorFlow/Keras (a su elección). Debe escribir el código desde cero o basarse en las documentaciones oficiales. Ejecute sus experimentos en Google Colab, Kaggle Notebooks o en su GPU local. Siéntanse libres de hacer uso de IA de forma educada, es decir, entendiendo realmente que lo que esté haciendo sea realmente lo que necesitan y que sobretodo lo entiendan.

1. Descargue el dataset, divídalo en Entrenamiento (70%), Validación (15%) y Prueba (15%).

Implemente Data Augmentation (rotaciones, flips, recortes) vital para evitar el sobreajuste en imágenes agrícolas.

2. Cargue los siguientes modelos pre-entrenados en ImageNet y congele sus capas base, reemplazando solo el cabezal de clasificación para nuestras 8 clases de mango:
 - a. ResNet (Puede usar ResNet50).
 - b. Inception (Puede usar InceptionV3).
 - c. MobileNet (Puede usar MobileNetV2 o V3-Small/Large).
3. Entrene los 3 modelos utilizando la misma función de pérdida (Cross-Entropy) y optimizador (ej. Adam) por un máximo de 15 a 20 épocas (o use Early Stopping).
4. Registre las métricas correspondientes para cada modelo:
 - a. Accuracy (Exactitud) en el conjunto de prueba.
 - b. F1-Score (Macro) en el conjunto de prueba.
 - c. Tamaño del modelo final (en Megabytes) al guardarlo en disco (.pth o .h5).
 - d. Tiempo de Inferencia: Cuántos milisegundos (ms) tarda en predecir una sola imagen (promedio sobre 100 imágenes).

Task 3

En la industria, el código es solo una herramienta; lo que el cliente paga es su criterio. Basado en los resultados de la Parte 2, redacte un dictamen ejecutivo de 1 a 2 páginas.

Inciso 1

Presente una tabla clara cruzando los 3 modelos versus las 4 métricas evaluadas (Accuracy, F1-Score, Tamaño en MB, Tiempo de Inferencia).

Inciso 2

Compare a ResNet e Inception frente a MobileNet. ¿Cuánto "Accuracy" sacrificó usted (si es que sacrificó algo) al usar MobileNet? ¿Cómo se correlaciona el tamaño en Megabytes con la arquitectura matemática que usted describió en la Parte 1?

Inciso 3

Responda al CEO de la startup: Considerando que los agricultores guatemaltecos usarán teléfonos con 2GB de RAM sin internet, ¿qué modelo exacto mandamos a producción y por qué? (Justifique por qué el modelo ganador es viable para Edge AI frente a los perdedores).